AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY UNDER SECTION 3 OF UGC ACT, 1956

SCHOOL OF COMPUTING CHENNAI

MACHINE LEARNING IN CYS
24CYS214

Assignment Report
Experimenting with Word Vectorizer Models

Submitted by : G.Venkatanatha
N.Adarsh Krishna

Part 1: Literature Review

1.1 Introduction to Word Vectorization

Natural Language Processing (NLP) requires transforming raw text into numerical representations that machine learning algorithms can process. This transformation is called word vectorization. Since computers inherently work with numbers rather than symbols, every word, sentence, or document must be encoded as a vector of real-valued numbers before any mathematical model can reason about it.

Word vectorization is foundational to virtually every modern NLP pipeline — from spam detection and sentiment analysis to machine translation and question answering. The choice of vectorization method significantly affects both the quality of downstream predictions and the computational efficiency of the system.

1.2 Bag of Words (BoW)

The Bag of Words model is the oldest and most intuitive approach to text vectorization. It represents a document as an unordered collection of its constituent words, completely ignoring grammar and word order. The model constructs a vocabulary of all unique tokens seen during training and then encodes each document as a fixed-length vector where each dimension corresponds to a vocabulary token.

Underlying Principle: Given a vocabulary V of size |V|, a document d is encoded as a vector v ∈ R^|V|, where v_i = count(word_i in d). The resulting matrix for a corpus is a term-document frequency matrix.

Strengths of BoW include simplicity, interpretability, and ease of implementation. Every feature has a clear meaning — the raw frequency of a specific word. However, it suffers from several shortcomings: it ignores word order and context entirely, produces extremely high-dimensional sparse vectors, and gives equal weight to common uninformative words (like "the", "is") and rare but meaningful ones.

Applications: BoW is well-suited for document classification, topic modelling, and spam filtering where word frequency patterns are strong discriminators.

1.3 TF-IDF (Term Frequency–Inverse Document Frequency)

TF-IDF is a statistical weighting scheme that refines the raw frequency counts of BoW by downweighting terms that appear frequently across many documents. The intuition is that a word appearing in almost every document carries very little

discriminative information, while a word that appears often in one document but rarely across the corpus is more informative.

Underlying Principle: For a term t in document d within corpus D:

$TF(t, d) = (\text{count of } t \text{ in } d) / (\text{total words in } d)$

$IDF(t, D) = \log(|D| / (1 + |\{d \in D : t \in d\}|))$

$TF\text{-}IDF(t, d, D) = TF(t, d) \times IDF(t, D)$

The IDF component penalises terms that appear in many documents. Words like "the", "and", or "is" — which appear everywhere — receive near-zero IDF scores and are effectively suppressed. This results in better feature discrimination compared to raw BoW.

Applications: TF-IDF is widely used in search engines (document ranking), text classification, and information retrieval. It remains one of the most reliable baselines for text classification tasks.

1.4 Hashing Vectorizer

The Hashing Vectorizer (also called the "hash trick" or feature hashing) applies a hash function directly to token strings to map them to a fixed-dimensional feature vector, without ever building an explicit vocabulary. This makes it extremely memory-efficient and fast.

Underlying Principle: A hash function h : words → {0, 1, ..., n-1} maps each word to an index in a fixed-size array of length n. The count (or frequency) is accumulated at that index. Multiple words may map to the same index — a phenomenon called hash collision — but with a large enough feature space (e.g., 2^18), collisions are rare.

Key advantages include: no need to store a vocabulary dictionary, constant memory usage regardless of corpus size, and dramatically faster vectorization. The trade-off is that feature indices are anonymous — you cannot recover which word corresponds to which dimension — making the model less interpretable. Collisions can also introduce a small amount of noise into the feature space.

Applications: Hashing Vectorizer is ideal for streaming data pipelines, very large corpora, online learning scenarios, and resource-constrained environments.

Part 2: Experimentation

2.1 Dataset: 20 Newsgroups

The 20 Newsgroups dataset is a well-known benchmark in text classification. It contains approximately 18,846 newsgroup posts organised into 20 categories spanning topics from computer hardware and sports to religion and politics. For this experiment, the dataset was split into two custom groups for binary classification:

- Group 1 — Technology, Sports & Space (10 categories): comp.graphics, comp.os.ms-windows.misc, comp.sys.ibm.pc.hardware, comp.sys.mac.hardware, comp.windows.x, rec.autos, rec.motorcycles, rec.sport.baseball, rec.sport.hockey, sci.space
- Group 2 — Religion, Medicine & Politics (10 categories): alt.atheism, soc.religion.christian, talk.religion.misc, sci.med, sci.crypt, sci.electronics, misc.forsale, talk.politics.guns, talk.politics.mideast, talk.politics.misc

The dataset is deliberately unbalanced: Group 1 contains 9,857 samples and Group 2 contains 8,989 samples. After an 80/20 train-test split (stratified), the training set has 15,076 documents and the test set has 3,770 documents.

2.2 How Each Vectorizer Processes This Dataset

Each vectorizer transforms the same preprocessed text into a numerical matrix, but through fundamentally different mechanisms:

Bag of Words on 20 Newsgroups: BoW scans all 15,076 training documents and builds a vocabulary of the 8,000 most frequent stemmed bigrams/unigrams. For each document, it counts how many times each vocabulary word appears. The result is a (15,076 × 8,000) sparse matrix. Words like "use", "get", "window", "file" — which are highly frequent in Group 1 — receive high counts. Words like "god", "peopl", "say" dominate Group 2. The classifier learns these frequency patterns directly.

TF-IDF on 20 Newsgroups: TF-IDF builds the same vocabulary but replaces raw counts with TF-IDF scores. A word that appears many times in one document but rarely elsewhere (highly specific technical jargon, for example) receives a high TF-IDF weight. Common words shared across both groups — like "would", "one", "know", "like" (184 overlapping words identified in the top 300) — receive very low IDF weights and are effectively suppressed. This gives Group-1-specific words (e.g., "svga", "vlb", "graphit") and Group-2-specific words (e.g., "atheism", "jesus", "medicin") more discriminative power.

Hashing Vectorizer on 20 Newsgroups: The Hashing Vectorizer never stores a vocabulary. Each stemmed word from the 18,846 documents is passed through the MurmurHash3 function and mapped to one of 8,000 bins. Both groups' documents are transformed in a single streaming pass. The same 8,000-dimensional feature space is produced, but without the memory overhead of a vocabulary dictionary. Words from the data exploration — including the noisy artefact "maxaxaxaxaxaxaxaxaxaxaxaxaxaxax" (3,318 occurrences in Group 1) — are simply hashed into some bin without special treatment.

2.3 Text Preprocessing Pipeline

All three vectorizers received identical preprocessed text. The preprocessing pipeline applied the following steps in order:

- Lowercasing: Converts all text to lowercase to treat 'Window' and 'window' identically.
- Special character removal: Strips numbers, punctuation, and non-alphabetic characters using regex [^a-zA-Z\s].
- Whitespace normalisation: Collapses multiple spaces into single spaces.
- Stopword removal: Removes 179 English stopwords from NLTK (e.g., 'the', 'is', 'at').
- Porter Stemming: Reduces words to their root form — 'running' → 'run', 'graphics' → 'graphit', 'people' → 'peopl'.

Preprocessing reduced the mean document length from 1,169.7 characters to 91.5 words. This compressed representation removes noise while preserving the core semantics needed for classification.

2.4 Experiment Configuration

Parameter	Value
Dataset	20 Newsgroups (all subsets, 18,846 samples)
Task	Binary classification: Group 1 vs Group 2
Train/Test Split	80% train (15,076) / 20% test (3,770), stratified
Max Features	8,000 (unigrams + bigrams, min_df=2)
Classifier	Logistic Regression (C=1.0, max_iter=1000)
Cross-Validation	3-fold on first 5,000 training samples
Preprocessing time	22.34 seconds for 18,846 documents

Part 3: Results & Analysis

3.1 Performance Results Summary

Method	Accuracy	Precision	Recall	F1-Score	Total Time (s)
Bag of Words (BoW)	0.8631	0.8507	0.8955	0.8725	8.56s
TF-IDF	0.8721	0.8551	0.9097	**0.8816 ★**	7.40s
Hashing Vectorizer	0.8523	0.8371	0.8910	0.8632	**2.59s ★**

★ = Best in category

3.2 Confusion Matrix Analysis

Method	True Negatives	False Positives	False Negatives	True Positives
Bag of Words	1,488	310	206	1,766
TF-IDF	1,494	304	178	1,794
Hashing Vectorizer	1,456	342	215	1,757

3.3 Visualizations

The chart below summarises the four key dimensions of the experiment: performance metrics across methods, time complexity breakdown, the performance-vs-time trade-off scatter plot, and the feature space comparison.

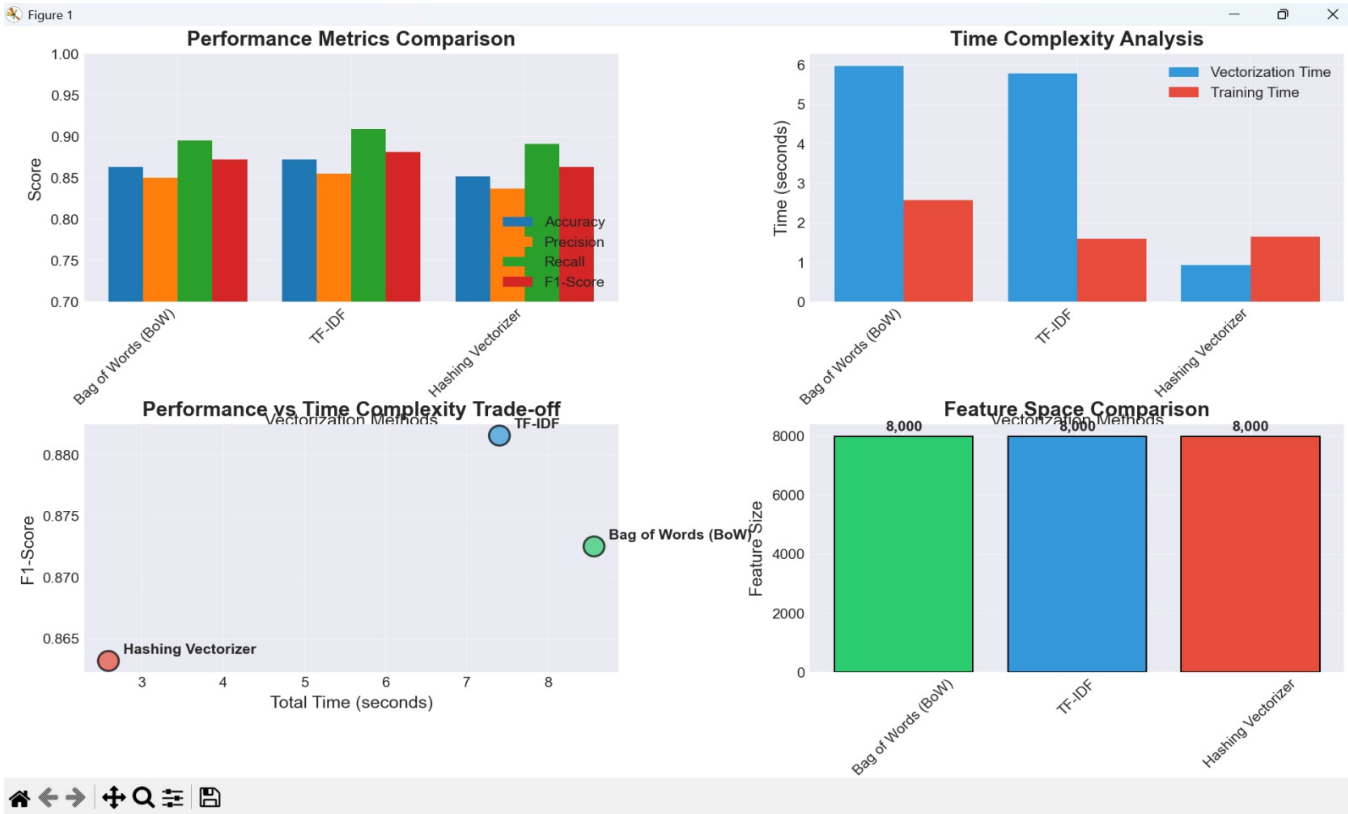

Figure 1: Vectorization Analysis — Performance, Time, Trade-off, and Feature Space

3.4 Why the Results Differ — In-Depth Analysis

3.4.1 Why TF-IDF Outperforms BoW

TF-IDF achieves the highest accuracy (0.8721) and F1-score (0.8816), outperforming BoW (0.8631 accuracy, 0.8725 F1) for a concrete and principled reason: intelligent feature weighting.

The data exploration revealed 184 overlapping words in the top 300 of both groups — words like "like", "one", "would", "know", "time", and "also". In a BoW representation, these highly frequent but non-discriminative words receive large positive weights simply because they appear often. The Logistic Regression classifier must then learn to suppress these noisy high-frequency features on its own, which is difficult and can overfit.

TF-IDF automatically suppresses these cross-group common words through the IDF component. A word appearing in 10,000 of 15,076 training documents receives an IDF close to zero, effectively removing it as a feature. This means the classifier receives a cleaner signal: high TF-IDF weights mark words that are both frequent in a document AND rare across documents — precisely the kind of domain-specific vocabulary that

distinguishes Group 1 ("svga", "vlb", "graphit", "orbit") from Group 2 ("atheism", "jesus", "medicin", "clinton").

The improvement is most visible in Recall: TF-IDF achieves 0.9097 vs BoW's 0.8955 — meaning TF-IDF correctly identifies more Group 1 documents that BoW misses (178 false negatives vs 206). This is consistent with better discriminative weighting.

3.4.2 Why Hashing Vectorizer Lags Behind Both

The Hashing Vectorizer scores lowest on all performance metrics (accuracy 0.8523, F1 0.8632), primarily due to two structural limitations:

- Hash collisions: With 8,000 bins and a large vocabulary, multiple words inevitably map to the same feature index. When two words with opposite class associations collide (e.g., a Group-1 word and a Group-2 word hash to the same bin), their signals cancel out, degrading classification quality.
- No IDF weighting: Unlike TF-IDF, the Hashing Vectorizer uses raw counts without any cross-document normalisation. The same problem that limits BoW — uninformative common words dominating the feature space — is present here too.

Despite its lower accuracy, the Hashing Vectorizer provides a compelling trade-off: its vectorization time is 0.93 seconds compared to ~6 seconds for BoW and TF-IDF — a 6x speedup. This is because it requires no vocabulary lookup table, no sorting, and no IDF computation. The total pipeline runs in just 2.59 seconds vs 7.40s (TF-IDF) and 8.56s (BoW).

3.4.3 Why BoW is Slower Than TF-IDF

Counterintuitively, BoW (8.56s total) is slower than TF-IDF (7.40s total) despite being conceptually simpler. The main reason is the training time: Logistic Regression on BoW features required 2.59s of training vs 1.61s for TF-IDF. This happens because TF-IDF features are normalised (L2-normalised by default), which produces better-conditioned feature matrices that the gradient descent solver in Logistic Regression converges on faster. Raw BoW counts are unnormalised integers that can have very different scales across features, slowing convergence.

3.4.4 Sparsity and Its Impact

All three methods produced matrices with approximately 99.3% sparsity — that is, roughly 99.3% of all matrix entries are zero. This is expected: no single document uses more than a tiny fraction of the 8,000-word vocabulary. Scikit-learn's sparse matrix representation (CSR format) handles this efficiently. The near-identical sparsity across

methods means this is not a differentiating factor; the performance and time differences arise purely from the vectorization scheme, not the storage format.

3.5 Trade-off Discussion

The scatter plot (bottom-left of Figure 1) captures the core insight: there is a clear trade-off between classification performance and computational speed. TF-IDF occupies the "best performance" position (F1 = 0.8816, ~7.4s), BoW falls in between on performance but is slowest (F1 = 0.8725, ~8.6s), and the Hashing Vectorizer is by far the fastest (F1 = 0.8632, ~2.6s).

For this dataset size (18,846 samples), all three methods complete within seconds, making the speed difference relatively minor. However, the pattern scales dramatically: for a corpus of 10 million documents, the Hashing Vectorizer's O(n) single-pass architecture and constant memory footprint make it the only practical choice. TF-IDF's O(n log n) IDF computation and vocabulary storage become bottlenecks at that scale.

If the priority is maximum accuracy on this task, TF-IDF is the clear winner. If the pipeline needs to be deployed in a real-time streaming system or on edge hardware, the Hashing Vectorizer's speed advantage justifies the ~2% drop in F1.

Part 4: Summary & Key Findings

4.1 Key Findings

- TF-IDF is the best overall method for this task, achieving F1 = 0.8816, the highest accuracy (0.8721), and the lowest false negative count (178). Its IDF weighting effectively suppresses the 184 overlapping common words between the two groups.
- Bag of Words is competitive (F1 = 0.8725) but lacks the intelligent suppression of common words. It is also the slowest method due to poor feature conditioning slowing the Logistic Regression solver.
- Hashing Vectorizer is dramatically faster (2.59s vs 7-9s) at the cost of a modest ~2% drop in F1. It is the best choice for large-scale or real-time applications.
- All three methods produced high-quality binary classifiers (accuracy > 85%) on this 18,846-sample corpus, confirming that the two topic groups are genuinely separable through vocabulary alone.
- A noisy artefact token — 'maxaxaxaxaxaxaxaxaxaxaxaxaxaxaxax' appearing 3,318 times — was present in Group 1 data. This kind of token highlights the importance of preprocessing; BoW and Hashing Vectorizer treated it as a regular high-frequency token, while TF-IDF's IDF weighting would have further downweighted it if it appeared in multiple documents.

GitHub Repository: https://github.com/GangavaramVenkatanatha/ML-Assginment

Source code, dataset loading scripts, and raw output logs are available at the repository above.